

Índice de Conteúdo: POO

1. Fundamentos da O.O

- Classe e Objeto (Instância)
- Métodos e Variáveis de Instância
- Métodos Construtores
- Exemplo de Código: Classe Jogador

2. Encapsulamento

- Modificadores de Acesso (Private)
- Setters e Getters
- Exemplo de Código: Classe Produto

3. Associação Todo-Parte e Coleções

- Agregação
- Composição
- Classe de Associação
- ArrayList
- Exemplo de Código: Classe Deck

4. Diagrama de Classe da UML

- Estrutura das 3 Partes (Nome, Atributos, Métodos)
- Simbologia de Visibilidade
- Representação de Agregação, Composição e Herança

5. Herança e Associação

- Superclasse e Subclasse
- Uso do Extends
- Sobrescrita (@Override)
- Polimorfismo
- Uso de super() e super.metodo()
- Exemplo de Código: Classe Personagem e Mago

6. Classes e Métodos Abstratos

- Conceito de Abstração
- Implementação Obrigatória
- Exemplo de Código: Classe Forma

7. Classes Interfaces

- Contrato de Comportamento
- Múltiplas Interfaces
- Exemplo de Código: Interface Jogavel

Guia de Estudo e Consulta:

Programação Orientada a Objetos (Java)

1. Fundamentos da O.O

A Programação Orientada a Objetos busca aproximar o mundo digital do mundo real, modelando sistemas através de "objetos" que possuem características e comportamentos próprios.

- **Classe:** É o **molde**, gabarito ou a planta baixa. Ela define o que um objeto do seu tipo deve ter (atributos) e o que ele deve fazer (métodos). A classe em si não ocupa espaço na memória para dados; ela apenas descreve a estrutura.
- **Objeto (Instância):** É a concretização da classe. Quando você usa a palavra-chave `new`, o Java aloca um espaço na memória para aquele objeto específico. Cada instância é única e possui sua própria identidade.
- **Variáveis de Instância (Atributos):** São as características ou o "estado" do objeto. Cada objeto tem sua própria cópia dessas variáveis. Se a classe for `Celular`, os atributos podem ser `marca`, `modelo` e `bateria`.
- **Métodos:** São os comportamentos ou ações. Eles definem a lógica de como o objeto interage com seus próprios dados ou com outros objetos.
- **Métodos Construtores:** São métodos especiais executados no momento da criação do objeto. Sua função principal é inicializar os atributos para que o objeto comece em um estado válido. Se nenhum construtor for escrito, o Java fornece um "construtor padrão" sem parâmetros.

Java

```
// Exemplo de Fundamentos
```

```
public class Jogador {  
    // Variáveis de instância  
    String nome;  
    int nivel;  
  
    // Método Construtor  
    public Jogador(String nome, int nivel) {  
        this.nome = nome;  
        this.nivel = nivel;  
    }  
  
    // Método de instância (comportamento)  
    public void subirNivel() {  
        this.nivel++;  
        System.out.println(nome + " subiu para o nível " + nivel);  
    }  
}
```

```
// No código principal:  
// Jogador j1 = new Jogador("Fabrício", 1); // Instanciação
```

2. Encapsulamento

O objetivo do encapsulamento é esconder os detalhes internos do funcionamento de um objeto e proteger seus dados contra acessos indevidos ou modificações acidentais que possam quebrar a lógica do sistema.

- **Private (Modificador de Acesso):** Ao declarar um atributo como `private`, garantimos que nenhuma outra classe possa ler ou alterar essa variável diretamente. Isso obriga o uso de métodos "porteiros".
- **Setters e Getters:** São a interface pública do objeto.
 - **Getter:** Método que retorna o valor de um atributo privado (acesso de leitura).
 - **Setter:** Método que recebe um valor e o atribui ao atributo privado (acesso de escrita). A grande vantagem é a **validação**: dentro do `set`, você pode colocar um `if` para impedir valores inválidos.

Java

```
public class Produto {  
    private double preco; // Atributo privado  
  
    // Getter  
    public double getPreco() {  
        return preco;  
    }  
  
    // Setter com validação  
    public void setPreco(double novoPreco) {  
        if (novoPreco >= 0) {  
            this.preco = novoPreco;  
        } else {  
            System.out.println("Erro: Preço não pode ser negativo!");  
        }  
    }  
}
```

3. Associação Todo-Parte e Coleções

Descreve como objetos se relacionam para formar estruturas mais complexas, onde um objeto "Todo" possui ou é composto por objetos "Parte".

- **Agregação:** É uma relação de "tem um", mas com independência. O "Todo" precisa da "Parte", mas se o "Todo" deixar de existir, a "Parte" continua existindo no sistema. (Exemplo: Um Time tem Jogadores. Se o time acabar, os jogadores ainda existem).

- **Composição:** É uma relação de dependência forte (vínculo vital). A "Parte" não existe sem o "Todo". (Exemplo: Uma Casa tem Cômodos. Se a casa for destruída, os cômodos também são). No código, o "Todo" geralmente cria a instância da "Parte" dentro de si.
- **Classe de Associação:** Usada quando uma relação Muitos-para-Muitos entre duas classes gera informações próprias que não cabem em nenhuma das duas (Exemplo: Aluno e Disciplina geram a classe "Nota").
- **ArrayList:** É uma coleção dinâmica da biblioteca `java.util`. Diferente de um Array comum (vetor), o `ArrayList` aumenta de tamanho automaticamente conforme adicionamos novos objetos, sendo a ferramenta ideal para implementar associações de "um para muitos".

Java

```
import java.util.ArrayList;

public class Deck {
    private String nome;
    // Agregação: O Deck possui uma lista de Cartas
    private ArrayList<Carta> cartas;

    public Deck(String nome) {
        this.nome = nome;
        this.cartas = new ArrayList<>(); // Inicializando a coleção
    }

    public void adicionarCarta(Carta c) {
        this.cartas.add(c);
    }
}
```

4. Diagrama de Classe da UML

A UML (Unified Modeling Language) é a representação visual da estrutura do sistema. No diagrama de classe, cada classe é um retângulo dividido em três compartimentos:

1. **Superior:** Nome da classe.
 2. **Intermediário:** Atributos (variáveis). Formato: visibilidade nome : tipo.
 3. **Inferior:** Métodos. Formato: visibilidade nome(parâmetros) : tipo_de_retorno.
- **Simbologia de Relações:**
 - **Agregação:** Linha com um **losango vazio** (branco) no lado do objeto "Todo".
 - **Composição:** Linha com um **losango preenchido** (preto) no lado do objeto "Todo".
 - **Herança:** Linha com uma **seta de triângulo vazio** apontando para a Superclasse.
-

5. Herança e Associação

Herança permite que uma classe (Subclasse) adquira todos os atributos e métodos de outra (Superclasse), promovendo o reuso de código e a especialização.

- **Superclasse e Subclasse:** A Superclasse é a mais genérica (ex: Veículo), enquanto a Subclasse é a mais específica (ex: Moto).
- **Extends:** Palavra-chave que estabelece o vínculo de herança.
- **@Override:** Anotação que indica que a Subclasse está "sobrescrevendo" um método da Superclasse para dar a ele um comportamento diferente.
- **Polimorfismo:** É a capacidade de uma variável de referência do tipo da Superclasse referenciar um objeto de qualquer uma de suas Subclasses e executar o método correto em tempo de execução.
- **Super() e Super.metodo():**
 - `super()`: Chama o construtor da classe pai (deve ser a primeira linha do construtor filho).
 - `super.metodo()`: Invoca a versão do método que está na classe pai, útil quando queremos aproveitar a lógica da pai e adicionar algo a mais.

Java

```
public class Personagem {
    protected int vida; // Protected permite que as filhas vejam

    public void atacar() {
        System.out.println("Ataque básico!");
    }
}

public class Mago extends Personagem {
    @Override
    public void atacar() {
        super.atacar(); // Faz o ataque básico da pai
        System.out.println("Lançando bola de fogo!"); // Adiciona algo novo
    }
}
```

6. Classes e Métodos Abstratos

Utilizados quando queremos criar um modelo de classe que sirva apenas como base para outras, mas que não faça sentido existir por si só.

- **Classe Abstrata:** Marcada com `abstract`. Você **não pode** criar um objeto dela (`new`). Ela serve apenas para ser herdada.
- **Método Abstrato:** Um método que não possui implementação (não tem as chaves `{}`). Ele define apenas a "assinatura". Isso **obriga** todas as subclasses a escreverem sua própria versão desse método.

Java

```
public abstract class Forma {
    // Método abstrato: toda forma tem área, mas o cálculo muda
    public abstract double calcularArea();
}

public class Quadrado extends Forma {
    private double lado;
    @Override
    public double calcularArea() {
        return lado * lado;
    }
}
```

7. Classes Interfaces

A interface é um "contrato" de comportamento. Ela define o que uma classe deve ser capaz de fazer, mas não como ela faz.

- Uma classe pode herdar de apenas uma classe pai, mas pode **implementar várias interfaces**.
- Todos os métodos em uma interface são, por padrão, públicos e abstratos.
- É ideal para garantir que classes de famílias totalmente diferentes possuam o mesmo conjunto de métodos (ex: Carro e Robô podem implementar a interface Controlavel).

Java

```
public interface Jogavel {
    void darPlay();
    void pausar();
}

public class Video implements Jogavel {
    @Override
    public void darPlay() {
        System.out.println("Vídeo rodando...");
    }

    @Override
    public void pausar() {
        System.out.println("Vídeo pausado.");
    }
}
```